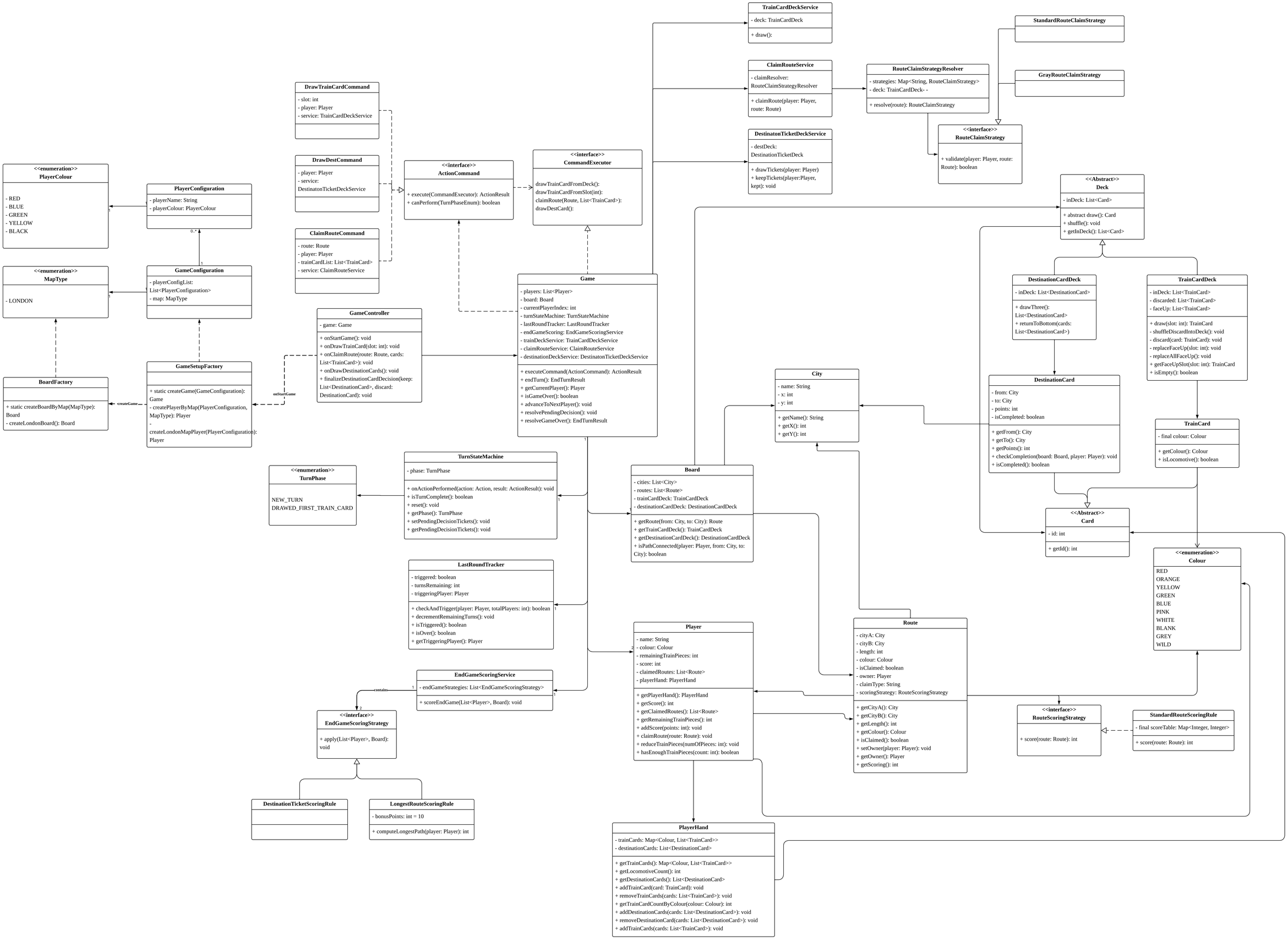

<<enumeration>>
PlayerColour
- RED
- BLUE
- GREEN
- YELLOW
- BLACK
PlayerConfiguration
- playerName: String
- playerColour: PlayerColour
<<enumeration>>
MapType
- LONDON
GameConfiguration
- playerConfigList: List<PlayerConfiguration>
- map: MapType
BoardFactory
+ static createBoardByMap(MapType): Board
- createLondonBoard(): Board
createGame
GameSetupFactory
+ static createGame(GameConfiguration): Game
- createPlayerByMap(PlayerConfiguration, MapType): Player
- createLondonMapPlayer(PlayerConfiguration): Player
onStartGame
DrawTrainCardCommand
- slot: int
- player: Player
- service: TrainCardDeckService
DrawDestCommand
- player: Player
- service: DestinatonTicketDeckService
ClaimRouteCommand
- route: Route
- player: Player
- trainCardList: List<TrainCard>
- service: ClaimRouteService
<<interface>>
ActionCommand
+ execute(CommandExecutor): ActionResult
+ canPerform(TurnPhaseEnum): boolean
<<interface>>
CommandExecutor
drawTrainCardFromDeck():
drawTrainCardFromSlot(int):
claimRoute(Route, List<TrainCard>):
drawDestCard():
GameController
- game: Game
+ onStartGame(): void
+ onDrawTrainCard(slot: int): void
+ onClaimRoute(route: Route, cards: List<TrainCard>): void
+ onDrawDestinationCards(): void
+ finalizeDestinationCardDecision(keep: List<DestinationCard>, discard: DestinationCard): void
Game
- players: List<Player>
- board: Board
- currentPlayerIndex: int
- turnStateMachine: TurnStateMachine
- lastRoundTracker: LastRoundTracker
- endGameScoring: EndGameScoringService
- trainDeckService: TrainCardDeckService
- claimRouteService: ClaimRouteService
- destinationDeckService: DestinatonTicketDeckService
+ executeCommand(ActionCommand): ActionResult
+ endTurn(): EndTurnResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ advanceToNextPlayer(): void
+ resolvePendingDecision(): void
+ resolveGameOver(): EndTurnResult
TrainCardDeckService
- deck: TrainCardDeck
+ draw():
ClaimRouteService
- claimResolver: RouteClaimStrategyResolver
+ claimRoute(player: Player, route: Route)
DestinatonTicketDeckService
- destDeck: DestinationTicketDeck
+ drawTickets(player: Player)
+ keepTickets(player:Player, kept): void
RouteClaimStrategyResolver
- strategies: Map<String, RouteClaimStrategy>
- deck: TrainCardDeck- -
+ resolve(route): RouteClaimStrategy
StandardRouteClaimStrategy
GrayRouteClaimStrategy
<<interface>>
RouteClaimStrategy
+ validate(player: Player, route: Route): boolean
<<Abstract>>
Deck
- inDeck: List<Card>
+ abstract draw(): Card
+ shuffle(): void
+ getInDeck(): List<Card>
DestinationCardDeck
- inDeck: List<DestinationCard>
+ drawThree(): List<DestinationCard>
+ returnToBottom(cards: List<DestinationCard>)
TrainCardDeck
- inDeck: List<TrainCard>
- discarded: List<TrainCard>
- faceUp: List<TrainCard>
+ draw(slot: int): TrainCard
- shuffleDiscardIntoDeck(): void
- discard(card: TrainCard): void
- replaceFaceUp(slot: int): void
- replaceAllFaceUp(): void
+ getFaceUpSlot(slot: int): TrainCard
+ isEmpty(): boolean
City
- name: String
- x: int
- y: int
+ getName(): String
+ getX(): int
+ getY(): int
DestinationCard
- from: City
- to: City
- points: int
- isCompleted: boolean
+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ checkCompletion(board: Board, player: Player): void
+ isCompleted(): boolean
TrainCard
- final colour: Colour
+ getColour(): Colour
+ isLocomotive(): boolean
<<Abstract>>
Card
- id: int
+ getId(): int
<<enumeration>>
Colour
RED
ORANGE
YELLOW
GREEN
BLUE
PINK
WHITE
BLANK
GREY
WILD
TurnStateMachine
- phase: TurnPhase
+ onActionPerformed(action: Action, result: ActionResult): void
+ isTurnComplete(): boolean
+ reset(): void
+ getPhase(): TurnPhase
+ setPendingDecisionTickets(): void
+ getPendingDecisionTickets(): void
<<enumeration>>
TurnPhase
NEW_TURN
DRAWED_FIRST_TRAIN_CARD
Board
- cities: List<City>
- routes: List<Route>
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck
+ getRoute(from: City, to: City): Route
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
+ isPathConnected(player: Player, from: City, to: City): boolean
LastRoundTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player
+ checkAndTrigger(player: Player, totalPlayers: int): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player
Player
- name: String
- colour: Colour
- remainingTrainPieces: int
- score: int
- claimedRoutes: List<Route>
- playerHand: PlayerHand
+ getPlayerHand(): PlayerHand
+ getScore(): int
+ getClaimedRoutes(): List<Route>
+ getRemainingTrainPieces(): int
+ addScore(points: int): void
+ claimRoute(route: Route): void
+ reduceTrainPieces(numOfPieces: int): void
+ hasEnoughTrainPieces(count: int): boolean
Route
- cityA: City
- cityB: City
- length: int
- colour: Colour
- isClaimed: boolean
- owner: Player
- claimType: String
- scoringStrategy: RouteScoringStrategy
+ getCityA(): City
+ getCityB(): City
+ getLength(): int
+ getColour(): Colour
+ isClaimed(): boolean
+ setOwner(player: Player): void
+ getOwner(): Player
+ getScoring(): int
EndGameScoringService
- endGameStrategies: List<EndGameScoringStrategy>
+ scoreEndGame(List<Player>, Board): void
contains
<<interface>>
EndGameScoringStrategy
+ apply(List<Player>, Board): void
<<interface>>
RouteScoringStrategy
+ score(route: Route): int
StandardRouteScoringRule
- final scoreTable: Map<Integer, Integer>
+ score(route: Route): int
DestinationTicketScoringRule
LongestRouteScoringRule
- bonusPoints: int = 10
+ computeLongestPath(player: Player): int
PlayerHand
- trainCards: Map<Colour, List<TrainCard>>
- destinationCards: List<DestinationCard>
+ getTrainCards(): Map<Colour, List<TrainCard>>
+ getLocomotiveCount(): int
+ getDestinationCards(): List<DestinationCard>
+ addTrainCard(card: TrainCard): void
+ removeTrainCards(cards: List<TrainCard>): void
+ getTrainCardCountByColour(colour: Colour): int
+ addDestinationCards(cards: List<DestinationCard>): void
+ removeDestinationCard(cards: List<DestinationCard>): void
+ addTrainCards(cards: List<TrainCard>): void